

Comparative Storage Format Benchmark

CSV, JSON, Parquet, and ORC — write performance, query performance, and partitioning strategy

Dataset	Kaggle <code>usdot/flight-delays</code> — <code>flights.csv</code> , 592.4 MB, 5,819,079 rows
Environment	PySpark 3.5.9, <code>local[2]</code> , Python 3.11.7
Query	<code>SELECT AIRLINE AS carrier, AVG(ARRIVAL_DELAY) AS avg_arr_delay FROM flights GROUP BY AIRLINE</code>

1. Format Coverage

CSV, JSON, Parquet, and ORC outputs were each written from the same source DataFrame and verified on disk.

2. Storage Size, Write Time, and Read Query Time

Format	Write Time (s)	Size (MB)	Data Files	Read Query Time (s)
CSV	9.39	605.80	5	14.26
JSON	17.05	2,736.44	5	29.01
Parquet	8.75	142.63	5	0.82
ORC	10.13	177.69	5	1.70

ANALYSIS

Parquet produced the smallest footprint (142.63 MB — roughly 23.5% of CSV and 5.2% of JSON), the fastest write (8.75s), and the fastest read query (0.82s, approximately 17x faster than CSV). ORC was a close second on both size (177.69 MB) and read speed (1.70s). CSV and JSON, being row-oriented and text-based, cannot prune columns at read time; Spark must parse the full row to satisfy a column-selective aggregation, which explains their larger footprints and slower queries. JSON's field-name repetition per record made it the least efficient format on every measured dimension.

3. Partition Control Experiment (Parquet)

Strategy	Data Files	Size (MB)	Write Time (s)
<code>repartition(20)</code>	20	159.37	25.19
<code>coalesce(2)</code>	2	143.03	8.74
<code>partitionBy("YEAR", "MONTH")</code>	16 (nested directories)	143.80	9.48

Disk output structure:

- `repartition(20)` — Forces a full shuffle to redistribute rows evenly across 20 partitions, producing 20 part-files in a single flat output directory. This incurs the highest write cost of the three strategies (25.19s) due to shuffle overhead, and results in the largest total output size.
- `coalesce(2)` — Merges existing partitions with minimal shuffling, producing 2 part-files in a single flat directory. This was the fastest (8.74s) and smallest (143.03 MB) of the three strategies in this run.
- `partitionBy("YEAR", "MONTH")` — Restructures the output into a nested directory hierarchy (`YEAR=.../MONTH=.../part-*.parquet`), with one subdirectory per year-month combination and 16 files total. Size and write time are close to `coalesce(2)` ; the structural advantage is not realized at write time but at query time, where a filter on `YEAR` or `MONTH` allows Spark to prune irrelevant partitions rather than scanning the full dataset.

Conclusion

For this dataset and query pattern, Parquet is the strongest overall choice, combining the smallest storage footprint with the fastest write and read performance; ORC is a close and viable alternative. CSV and JSON remain useful for interoperability but are not suited to analytical workloads at this scale. On partitioning, `coalesce(2)` minimizes both file count and write cost, `repartition(20)` trades additional write cost for more evenly distributed files, and `partitionBy("YEAR", "MONTH")` trades a marginally more complex output layout for partition pruning on downstream, year/month-filtered queries.